

姓名：邱翊碩

系級：航太系 117

學號：F44134057

作業：作業四

主程式檔案名稱：hw4.java

其他檔案：

Card.java

priest.java

tank.java

shooter.java

assassin.java

Fightground.java

Player.java

Game_command.java

Updater.java

saving data (txt. documents) :

player_info.txt

playground.txt

一. 程式需求描述：

A. 基本功能：實作基本遊玩環境

1. 資料讀取與初始化：程式需能從存檔 (`player_info.txt`, `playground.txt`) 讀取玩家的初始生命值、能量、手牌和戰場設定。
2. 遊戲進行：完整執行遊戲規定的回合制戰鬥，並在回合內執行佈局、技能、攻擊、判定四個階段。
3. 玩家與卡牌資訊追蹤：追蹤並更新玩家的生命值、能量值（每回合回復至當前回合數）和卡牌在戰場放置區的狀態。
4. 卡牌機制設定：設計四種卡牌角色：牧師 (Priest)、坦克 (Tank)、射手 (Shooter)、刺客 (Assassin)，每種角色有獨特的技能。

B. 設計方面

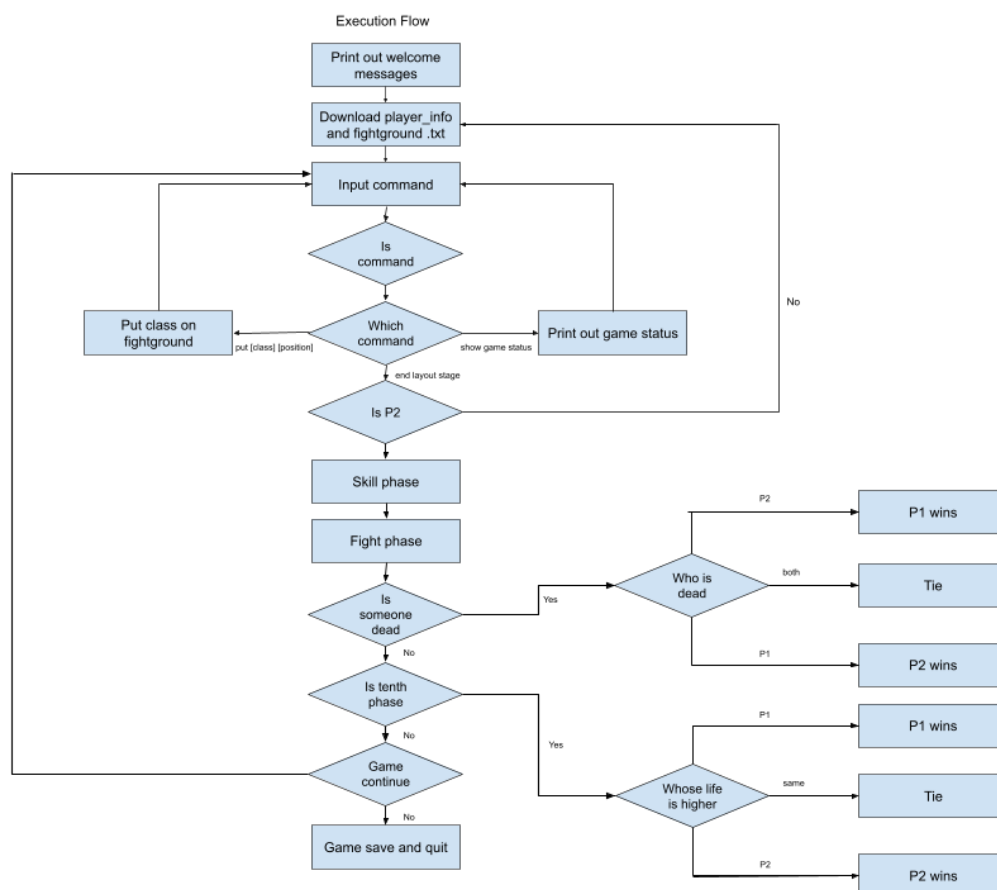
1. 本程式的撰寫大綱大致分成：資料讀入讀出、戰鬥流程、職業卡片的 `attribute` 和方法、資料的儲存、指令判讀，每個部分大致上都由一個 `.java` 檔負責，分工明確。
2. 將第一點的功能撰寫並整理完後以線性的方式寫入主程式
3. 本程式中 `Player.java`、`Fightground.java` 與其他類別採用封裝蓋念，確保 `private` 的屬性（玩家生命值、能量、手牌等等），外部只能透過類別的 `method` 來進行操作。
4. 利用物件之間的關係進行資料的處理，建立 `Player.java` 存放玩家資料，`Fightground.java` 用來存放場地資料，並互相與主程式 `hw4.java` 交換程式中的卡牌 `object`，以此對卡牌 `object` 進行管理。
5. 採用繼承與多型來設計卡牌體系，以 `Card.java` 為 `super class`，其他職業(如 `priest.java`)設為 `sub class`，確保卡牌 `method` 可以統一被呼叫。

C. 可能吸引人的點

1. 類別分工、物件關係明確
2. 主程式流程呈線性容易理解
3. 變數的命名明確，可以較清楚知道變數的用途
4. 程式包含大量註解，輔助閱讀
5. 不會因為 `Exception` 產生遊戲的中斷
6. 建立一定程度的函數庫，主程式中沒有過多多餘的重複內容

二. 程式流程

主程式流程圖.



a. 主程式執行順序文字解說

第一步. 先列出歡迎訊息與指令列

第二步. 從.txt 檔案載入初始遊戲資訊

第三步. 佈局階段接收指令，此階段會分析接收到的指令並執行相對應的功能，指令為 **put** 時放置指定戰鬥單位，**show game status** 顯示遊戲資訊，**end layout stage** 結束該玩家回合，如果玩家不為 **P2** 則再回到第三步一開始，由 **P2** 繼續佈局

第四步. 待佈局階段完結後，開始技能階段，結束時顯示遊戲資料

第五步. 待技能階段完結後，開始戰鬥階段

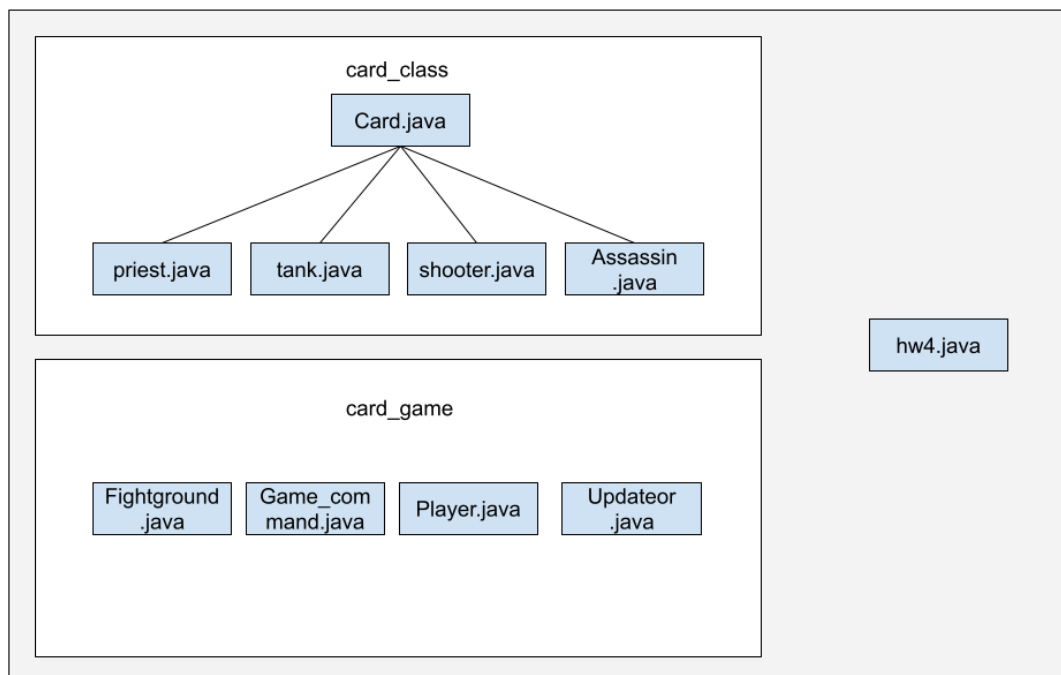
第六步. 檢視雙方是否存活已決定誰獲得勝利，如果此回合為第十回

合，則會對玩家血量多寡進行勝負判定，如果皆為滿足上述情況則近下一步操作。

第七步. 如果勝負沒有結束，詢問使用者是否繼續遊戲，如果否則儲存遊戲

三. Object / Class 描述

程式的簡易關係



Part 1.

Part1 中的類別主要目的為建置每個基本卡片的特徵與方法

1. Card :

設變數為 **protected** 以方便後續繼承的使用，技能優先度為技能的發動順序，1 為最快如 **priest**，4 為最慢如 **assassin**

Attribute :

protected int Max_life 最大生命值

protected int life 目前生命值

protected int atk 攻擊力

protected int Action_speed 技能優先度

Method :

void skill() 空技能(無意義，單純表現共通特質)

void assign_life() 回傳生命

void direct_attack() 攻擊主戰者

void card_damage() 攻擊對方卡片

static void summon() 生成卡片物件給 **Fightground object**

int get_action_speed() 回傳技能優先度

int get_atk() 回傳攻擊力

int get_life() 回傳生命

Card 為所有職業類別的母 **class**，宣告與定義卡片的共通 **attribute** 與 **method**，如 **.life**、**.get_life()**與攻擊的 **card_attack()**，讓子 **object** 可以攻擊其他的 **object** 與攻擊 **Player object**，在選擇攻擊主戰者或卡片時會用此 **Fightground object** 的格子是否為 **null** 判斷，該 **class** 並不會直接拿來生成 **object**。

2. priest (extends Card) :

Attribute :

int static pp_consume 能量消耗(屬於 **class**)

Method :

void skill() 回復我方 **Card object 10 life**

priest() **constructor**，設置預設 **priest** 屬性的初始值

priest (int, int) **constructor**，設置存檔讀取的初始值

String toString() 回傳” **priest**”

int get_pp_consume() 回傳 **priest** 的 **pp consume**

Card 的子 Class，object 有自己的 attribute 的值與獨有的全體治療 skill()，會幫其他的 Card object 回 life 值，該 Class 的 object 在程式中會被生成。

3. tank (extends Card) :

Attribute :

int static pp_consume 能量消耗(屬於 class)

Method :

void skill() 回復自身 20 life

tank() constructor，設置預設 tank 屬性的初始值

tank (int, int) constructor，設置存檔讀取的初始值

String toString() 回傳"tank"

int get_pp_consume() 回傳 tank 的 pp consume

Card 的子 Class，object 有自己的 attribute 的值與獨有的自我治療 skill()，該 Class 的 object 在程式中會被生成。

4. shooter (extends Card) :

Attribute :

int static pp_consume 能量消耗(屬於 class)

Method :

void skill() 攻擊對方全體 Card object 10 life

shooter () constructor，設置預設 shooter 屬性的初始值

shooter (int, int) constructor，設置存檔讀取的初始值

String toString() 回傳"shooter"

int get_pp_consume() 回傳 shooter 的 pp consume

Card 的子 Class，object 有自己的 attribute 的值與獨有的全體攻擊 skill()，會對其他的全體 Card object 發動攻擊，該 Class 的 object 在程式中會被生成。

5. assassin (extends Card) :

Attribute :

int static pp_consume 能量消耗(屬於 class)

Method :

void skill() 攻擊敵方主戰者 50 life

assassin () constructor，設置預設 shooter 屬性的初始值

assassin (int, int) constructor，設置存檔讀取的初始值

String toString() 回傳" assassin"

int get_pp_consume() 回傳 assassin 的 pp consume

Card 的子 Class，object 有自己的 attribute 的值與獨有的直接攻擊主戰者的 skill()，該 Class 的 object 在程式中會被生成。

Part 2

Part2 包含多種在主程式中會用到的但較為複雜的功能，為了實現主程式的線性流程，每個 class 都有自己主要明確的分工，每個 object 都有一定的獨立性。

1. Fightground :

Attribute :

int Phase 儲存當前回合數

Card P1_fight_area[] 儲存 P1 佈署的子 Card object

Card P2_fight_area[] 儲存 P2 佈署的子 Card object

Method :

String ground_status() 回傳場地資料字串

Card[] get_P1_fight_area() 回傳 P1 Card P1_fight_area[]

Card[] get_P2_fight_area() 回傳 P2 Card P2_fight_area[]

void stage_update() 清除 life <= 0 的卡片 object

void assign_Phase() 設定新的 Phase 值

int get_Phase() 回傳 Phase 值

object 在主程式一開始會被生成，Fightground object 主要功能為儲存在戰場上的 Card objects，另外有 ground_status() 可以回傳場地狀態。在此 Fightground object 的 Card objects 會被呼叫自己的攻擊與技能並在 Fightground object 進行技能與戰鬥階。

2. Game_command:

Method :

static void welcome_and_explain() 印出歡迎訊息

void command_execute() 執行解析後的指令

int command_analysis() 解析指令指令為 put 回傳 1，

show_game_status 回傳 2，**end_layout_stage** 回傳 3

void put() 將指定單位放到指令位置

void show_game_status() 印出目前遊戲資訊

void end_layout_stage() 結束回合

Game_command 這個 class 主要功能為處理玩家所輸入的指令並執行，裡面的方法皆為 static 且無自己的 attribute，所以不會生

成 object 在主程式。

3. Player :

Attribute :

int ID 記錄此玩家為 **P1** 還是 **P2**，應用於多型的狀況

int life 玩家生命

String player_name 玩家名子

int pp 玩家能量

int hand_area[] 玩家手牌，[0]代表 **priest**, [1] 代表 **tank**,
[2] 代表 **shooter**, [3] 代表 **assassin**

Method :

Player() **Constructor** 設定 ID

String toString() 回傳玩家基本訊息

void assign_life() 設定目前血量

void assign_pp() 設定目前能量

String get_name() 回傳名子

int get_life() 回傳生命

int[] get_hand_area() 回傳手牌數

int get_id() 回傳 ID

int get_pp() 回傳能量

boolean is_alive() 回傳是否存活

object 在主程式一開始會被生成(P1 和 P2)，用於儲存玩家的基本資料，如 life、pp(energy)、ID、player_name 與 hand_area 等相關資訊，除了回傳資訊的 method 另有方法.is_alive()可以判斷玩家生命是否歸 0。此 Player object 會被 Card 的子 object 攻擊，pp 值會與 Card 子 object 的 pp_consume 相減。

4.Updator :

Attribute :

static boolean Playerdata_download_succeed

static boolean Playground_download_succeed

兩個 **attributes** 皆用於判斷資料是否上傳成功

Method :

static boolean save_is_downloaded() 確定資料上傳成功

static void download_Player_data() 下載玩家資料

static void download_Fightground_data() 下載場地資料

static void upload_Player_data() 上傳玩家資料

static void upload_Fightground_data 上傳場地資料

Game_command 這個 class 主要功能為處理初始資料的讀取與最終資料的上傳，方法全部為 **static**，不會生成這個 class 的 **objec**。

四. 使用說明

a. 本系統為一個簡易的回合制卡牌對戰遊戲，以下為遊戲流程的簡述：

1. 載入玩家資料與戰場配置
2. 每回合進行卡牌的召喚佈署(佈局階段)
3. 依照技能速度 **Action Speed** 進行技能階段(技能階段)
4. 依序進行攻擊階段(攻擊階段)
5. 回合數來到 10 回合，或直到一方生命歸零時會結束遊戲並宣布勝負
6. 每個回合結束時選擇是否存檔

註 1. 所有玩家生命值、手牌卡片數量、場上卡片會更新到 `player_info.txt` 與 `playground.txt`

註 2. 若遊戲結束後選擇存檔，程式會自動更新所有資料後退出。若選擇不存檔則直接進入下一回合，遊戲並不會在玩家選擇繼續遊戲自動保存每個回合，還請玩家特別留意。

註 3. 欲查閱遊戲規則請參考 `readme.txt` 說明第 3 點

b. 以下為使用本系統的方法

1. 在收放 hw4 的資料夾打開 cmd，並在 cmd 裏頭編譯 hw4
ex. `javac hw4.java`
2. 執行 hw4，還請使用者另外輸入兩個參數 `player_info.txt`
`playground.txt`(儲存檔案)
ex. `hw4 player_info.txt playground.txt`
3. 程式將自動讀取存檔後開始並顯示遊戲畫面與指令說明，請玩家依照提示輸入指令
4. 程式會依指令結果自動進入技能階段、戰鬥階段，並更新場地

c. 使用上的須知

1. 所有操作皆以指令的形式輸入，請依照畫面提示的格式輸入
2. 程式會根據輸入內容進行分析，若格式錯誤、空白或逗號位置不正確，系統會提示錯誤

3. 請確認所有檔案都放在同一個資料夾底下，並且該資料夾內包含：
player_info.txt、playground.txt
4. 程式啟動時會自動讀取 player_info.txt、playground.txt 的檔案，還請使用者利用此這兩個檔案來設定遊戲初始狀態與資料
5. 程式所有輸出皆為英文
6. 以下為 player_info.txt、playground.txt 的格式與各數字與文字的代表意義，請使用者在設定資料時參考此格式，若資料不完整或錯誤將無法開始遊戲

player_info.txt:

7

Jacky,700,6

1,0,0,0

Emily,500,6

0,0,1,0

7 (回合數)

Jacky,700,6 (玩家一名稱), (玩家一生命值), (玩家一能量點)

1,0,0,0 (玩家一牧師持有數), (玩家一坦克持有數), (玩家一射手持有數), (玩家一刺客持有數)

Emily,500,6 (玩家二名稱), (玩家二生命值), (玩家二能量點)

0,0,1,0 (玩家二牧師持有數), (玩家二坦克持有數), (玩家二射手持有數), (玩家二刺客持有數)

playground.txt:

Jacky

1,assassin,50,70

2,shooter,80,100

3,tank,200,60

Emily

1,priest,60,50

2,none

3,tank,200,60

Jacky (玩家一名稱)

1,assassin,50,70 (區域一), (卡片), (生命值), (攻擊力)

2,shooter,80,100 (區域二), (卡片), (生命值), (攻擊力)

3,tank,200,60 (區域三), (卡片), (生命值), (攻擊力)

Emily (玩家二名稱)

1,priest,60,50 (區域一), (卡片), (生命值), (攻擊力)

2,none (區域二), (空格)

3,tank,200,60 (區域三), (卡片), (生命值), (攻擊力)

註.區域為不可變參數，請勿更改區域順序

d. 遊戲規則與流程

1. 遊戲目標：

玩家需在 10 回合內將對手生命歸零；若超過 10 回合仍未分出勝負，則比較生命值高低決定勝負。若生命相同，則為平手。

- 任一玩家生命歸零 → 另一方獲勝
- 雙方在同一回合同時死亡 → 平手
- 回合數到達 10 → 比較雙方生命決定勝負
- 若生命相同 → 平手

2. 玩家資訊：

每位玩家擁有：

- 生命值
- 能量
- 手牌(最多三張)

註. 回合開始時，玩家的能源值會自動設定為當前回合和數

3. 卡片種類

遊戲共有四種卡片，每種卡片擁有不同生命、攻擊力與技能：

priest

- Life: 200
- ATK: 50
- 召喚能量消耗：2
- 技能：回復己方全體生命

tank

- Life: 300
- ATK: 60

- 召喚能量消耗：2
- 技能：回復自身生命

shooter

- Life: 150
- ATK: 100
- 召喚能量消耗：3
- 技能：對敵軍全體造成 10 點固定傷害

assassin

- Life: 150
- ATK: 70
- 召喚能量消耗：3
- 技能：直接對敵方玩家造成 50 點固定傷害

註. 在技能階段時，卡片技能發動優先順序為 priest --> tank --> shooter --> assassin

4. 回合流程

每回合包含三個主要階段：

階段 1：佈局階段

玩家會在此階段可以進行卡片召喚、顯示資料與結束回合的動作，此三項動作皆以指令方式使用，佈局玩家行動順序：P1 → P2

- put [class] [position]：將一張卡片召喚到指定位置(1~3)
- show game status：顯示玩家資訊與戰場狀態
- end layout stage：結束本階段，進入技能階段

註.召喚限制：

- 玩家需持有該類卡片
- 玩家能量須足夠
- 位置不得被佔用

階段 2：技能階段(Skill Phase)

技能階段由系統自動執行，不需玩家輸入，技能玩家行動順序：P1 → P2。技能執行由 priest --> tank --> shooter --> assassin 排序來發動技能，若有卡片死亡會直接移除該卡

階段 3：戰鬥階段(Battle Phase)

技能階段結束後開始攻擊階段，佈局玩家行動順序：P1 → P2

攻擊規則：

- 若雙方在同一位置皆有卡牌 → 互相攻擊
- 若敵方位置為空 → 直接攻擊玩家，扣除生命值每次攻擊後，如卡片生命為 0，則移除該卡。

5. 存檔說明

每回合結束後系統會詢問是否儲存進度：

輸入 Y：

- 更新 `player_info.txt`
- 更新 `playground.txt`
- 結束遊戲

輸入 N：

- 進入下一回合

五. 指令範例與其他補充說明：

以下為程式指令範例

a. `put [class name] [position]`

佈署手上的單位至指定位置

ex. `put assassin 1`

b. `show game status`

顯示當前的場地與戰鬥資料，包含：回合數、玩家生命、能量，玩家手牌擁有卡片列表與戰場三格區域卡片資訊

ex. `show game status`

c. `end layout stage`

結束自己的佈署階段，進入下個階段

ex. `show individual score Alice`

d. Y / N (在存檔階段使用)

選擇是否要存檔，Y 為存檔，N 為繼續遊戲，此指令在佈局階段不可使用

ex. Y

e. 補充說明：

1. 若格式正確，會顯示「**command success**」；若格式錯誤，則會顯示相對應錯誤說明
2. 程式設有例外處理，避免因輸入錯誤導致崩潰
3. 程式本身用英文表示，如果對於某項功能有疑問可以再參考本文件